

LECTEUR RADIO ET PROGRAMMATION

Un système de diffusion radio automatisé pour une expérience d'écoute optimale

Documentation - Lecteur Radio et Programmation

Vue d'ensemble

Ce projet implémente un système de diffusion radio automatisé avec deux composants principaux :

- **Programmation** : Gestion de la programmation horaire des séquences audio
- **Player** : Lecteur audio avec buffers préchargés pour une diffusion continue

Classe Programmation

Description

La classe Programmation gère la programmation horaire des séquences audio. Elle permet de définir des créneaux horaires précis (HH:MM) et de persister ces programmations dans une base de données SQLite.

Initialisation

```
from programmation import Programmation
```

```
# Création avec paramètres par défaut  
prog = Programmation()
```

```
# Avec dossier source personnalisé  
prog = Programmation(source_folder="/path/to/audio/files")
```

```
# Avec base de données personnalisée  
prog = Programmation(db_path="/path/to/programmation.db")
```

Méthodes principales

Gestion de la bibliothèque audio

- `reload_library()` : Recharge la bibliothèque depuis le dossier source
- `configure_source_folder(folder_path)` : Change le dossier source des fichiers audio

Gestion de la programmation

- `set_daily_schedule(daily_schedule: Dict[int, List[str]])` : Définit la grille horaire complète
- `get_sequence_for_time(time_key: Union[int, str])` : Récupère la séquence pour un horaire donné
- `set_sequence_for_time(time_key: Union[int, str], files: List[str])` : Définit la séquence pour un horaire
- `save_schedule()` : Sauvegarde la programmation en base de données

Lecture automatique

- `get_next_event()` -> `List[str]` : Retourne la prochaine séquence et avance la programmation
- `peek_next_event()` -> `List[str]` : Aperçu de la prochaine séquence sans avancer
- `reset(start_time: Optional[datetime] = None)` : Réinitialise la programmation à un instant donné

Utilitaires

- `get_schedule_times()` -> `List[str]` : Liste des horaires programmés (format HH:MM)

Format des horaires

Les horaires sont représentés en minutes depuis minuit :

- 0 = 00:00
- 60 = 01:00
- 90 = 01:30
- 1439 = 23:59

Ou au format chaîne "HH:MM" :

- "00:00", "01:00", "23:59"

Exemple d'utilisation

```
prog = Programmation()
```

```
# Configurer le dossier source
```

```
prog.configure_source_folder("/home/user/audio")
```

```
# Définir une séquence à 9h00
```

```
prog.set_sequence_for_time("09:00", ["/path/to/file1.mp3", "/path/to/file2.wav"])
```

```
# Sauvegarder
```

```
prog.save_schedule()
```

```
# Obtenir la prochaine séquence
```

```
next_sequence = prog.get_next_event()
```

Classe Player

Description

La classe Player gère la lecture automatique des séquences audio avec deux buffers préchargés pour assurer une transition fluide entre les séquences.

Initialisation

```
from player import Player
```

```
from programmation import Programmation
```

```
prog = Programmation()
```

```
player = Player(prog, volume=80) # Volume à 80%
```

Méthodes principales

Contrôle de lecture

- `start()` : Démarre la lecture automatique
- `stop()` : Arrête la lecture
- `next()` : Passe immédiatement à la séquence suivante
- `automation_start()` : Alias pour `start()`

Gestion des séquences forcées

- `force_next_sequence(sequence: List[str])` : Force la lecture d'une séquence personnalisée

Informations sur l'état

- `get_current_title()` -> str : Titre du fichier en cours
- `get_next_title()` -> str : Titre du prochain fichier
- `get_next_sequence()` -> List[str] : Liste des fichiers de la prochaine séquence
- `get_remaining_time()` -> int : Temps restant du fichier actuel (ms)
- `get_buffer_status()` -> List[int] : État des buffers [buffer1_size, buffer2_size]

Architecture des buffers

Le lecteur utilise deux buffers :

- **Buffer 0** : Séquence en cours de lecture
- **Buffer 1** : Séquence suivante, préchargée en RAM

Quand la séquence du buffer 0 se termine, elle est remplacée par le buffer 1, et un nouveau buffer 1 est chargé.

Extensions audio supportées

- MP3 (.mp3)
- WAV (.wav)
- FLAC (.flac)
- OGG (.ogg)
- M4A (.m4a)

Validation audio

Chaque fichier est validé avec librosa avant d'être ajouté aux buffers. Les fichiers corrompus ou non reconnus sont ignorés avec un message d'erreur.

Exemple d'utilisation

```
# Initialisation
```

```
prog = Programmation()
```

```
player = Player(prog)
```

```
# Démarrage automatique
```

```
player.start()
```

```
# Forcer une séquence personnalisée
```

```
player.force_next_sequence(["/path/to/special.mp3", "/path/to/ad.wav"])
```

```
# Obtenir l'état
```

```
current = player.get_current_title()
```

```
remaining = player.get_remaining_time()
```

```
buffers = player.get_buffer_status()
```

Interfaces graphiques

PlayerGUI

Interface pour contrôler le lecteur :

- Boutons Démarrer/Arrêter/Suivant
- Contrôle du volume
- Affichage du statut de lecture
- État des buffers
- Historique des actions

ProgrammationGUI

Interface pour gérer la programmation :

- Sélection d'horaire (HH:MM)
- Gestion des fichiers par créneau
- Sauvegarde automatique
- Statistiques de programmation

Base de données

Structure

Table sequences :

- start_time (TEXT PRIMARY KEY) : Horaire au format "HH:MM"
- files (TEXT) : Liste JSON des chemins de fichiers

Persistance automatique

Toutes les modifications de programmation sont automatiquement sauvegardées dans la base de données.

Gestion d'erreurs

Player

- Fichiers introuvables : Signalés dans error_record
- Fichiers audio invalides : Ignorés avec message d'erreur
- Timeouts de lecture : Arrêt automatique après 10 minutes

Programmation

- Chemins invalides : Convertis en chemins absolus
- Horaires invalides : Validation automatique
- Base de données corrompue : Reconstruction automatique

Architecture threadée

Player

- Thread principal : Interface utilisateur
- Thread de lecture : _play_loop() avec buffers synchronisés
- Verrouillage : threading.Lock() pour accès sécurisé aux buffers

Sécurité

- Threads daemon : Arrêt automatique à la fermeture du programme
- Gestion d'événements : Communication thread-safe entre composants

Dépendances

- vlc : Lecture audio
- librosa : Validation des fichiers audio
- tkinter : Interfaces graphiques
- sqlite3 : Persistance des données
- pathlib : Gestion des chemins de fichiers

Utilisation typique

1. **Configuration initiale**
2. `prog = Programmation()`
`prog.configure_source_folder("/path/to/audio")`
3. **Programmation**
4. `prog.set_sequence_for_time("08:00", ["morning_show.mp3"])`
`prog.set_sequence_for_time("12:00", ["news.mp3", "weather.wav"])`
`prog.save_schedule()`
5. **Diffusion**
6. `player = Player(prog)`
`player.start() # Diffusion automatique selon la programmation`

Notes techniques

- **Performance** : Buffers préchargés pour transitions instantanées
- **Fiabilité** : Validation audio et gestion d'erreurs robuste
- **Extensibilité** : Architecture modulaire avec interfaces définies
- **Persistance** : Sauvegarde automatique des modifications
- **Interface** : Contrôles intuitifs pour gestion radio professionnelle